

Polar H10 蓝牙ECG数据格式技术规范

摘要

本文档基于对bleakheart开源库的深入分析，详细描述了Polar H10胸带设备通过蓝牙低功耗(BLE)传输ECG数据的完整格式规范。通过分析核心Python实现代码，我们获得了ECG数据帧的精确结构、解码方法和实际应用指南。

1. 概述

Polar H10是一款支持ECG测量的专业级胸带式心率监测设备。它通过Polar专有的Measurement Data (PMD)接口提供实时ECG数据流，采用蓝牙低功耗技术进行数据传输。

1.1 技术特性

- 采样频率: 130 Hz (默认)
- 数据分辨率: 14位
- 数据精度: 微伏级别 (μV)
- 传输协议: BLE (蓝牙低功耗)
- 数据编码: 小端序有符号整数

2. BLE服务架构

2.1 服务UUID

Polar H10使用专有的PMD服务进行ECG数据传输：

服务名称: Polar Measurement Data (PMD)
控制特征值: FB005C81-02E7-F387-1CAD-8ACD2D8DF0C8
数据特征值: FB005C82-02E7-F387-1CAD-8ACD2D8DF0C8

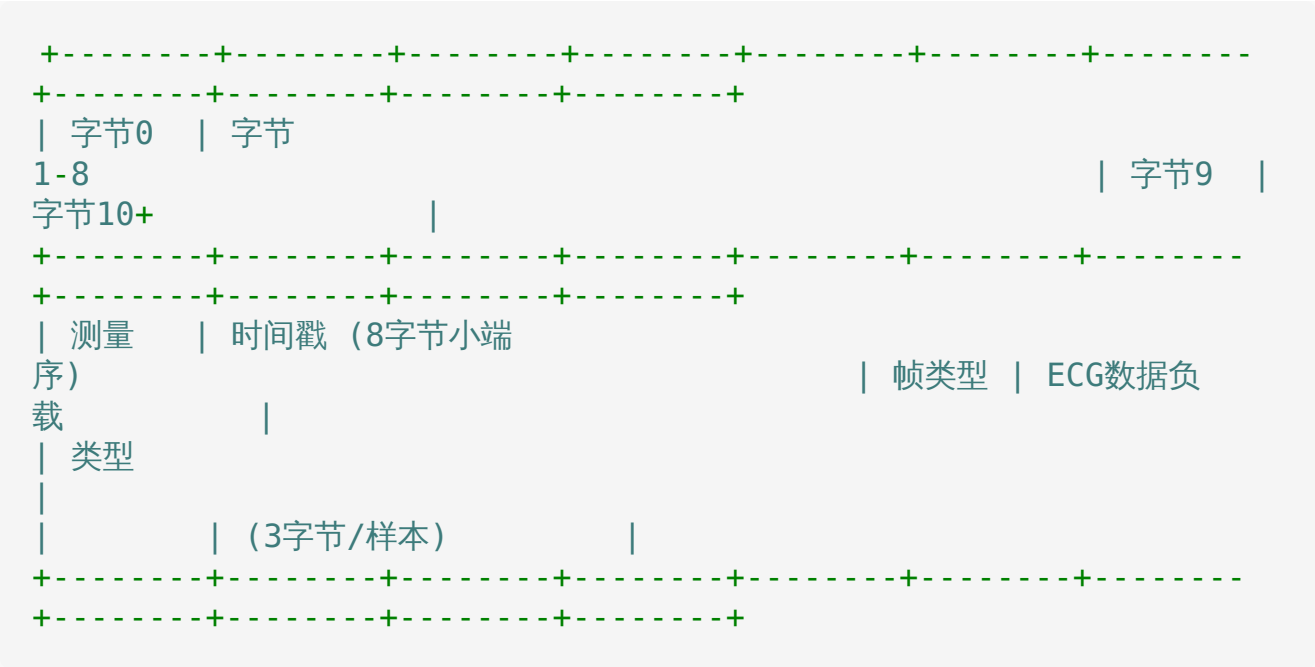
2.2 通信流程

- 连接建立:** 通过BLE连接到Polar H10设备
- 服务发现:** 查找PMD服务和特征值
- 参数查询:** 通过控制特征值查询ECG可用参数
- 开始流传输:** 发送START命令开始ECG数据流
- 数据接收:** 通过数据特征值接收ECG数据帧

6. 停止流传输: 发送STOP命令停止数据流

3. ECG数据帧格式

3.1 帧结构概览



3.2 详细字段说明

字节0: 测量类型

值: 0x00 (固定值, 表示ECG测量)

字节1-8: 时间戳

长度: 8字节
格式: 小端序无符号64位整数
单位: 传感器内部时间单位
说明: 指向数据帧中最后一个ECG样本的时间

字节9: 帧类型

值: 0x00 (ECG数据帧的固定值)

字节10+: ECG数据负载

格式：连续的3字节ECG样本
编码：小端序有符号24位整数
单位：微伏 (μV)
约束：总长度必须是3的倍数

3.3 ECG样本编码

每个ECG样本使用3字节表示：

样本字节布局（小端序）：

```
+-----+-----+-----+
|  LSB   |  MID   |  MSB   |
+-----+-----+-----+
| 字节0   | 字节1   | 字节2   |
+-----+-----+-----+
```

解码算法：

```
def decode_ecg_sample(bytes_3):
    """解码单个ECG样本"""
    value = int.from_bytes(bytes_3, 'little', signed=True)
    return value # 单位：微伏
```

编码范围：- 最小值: $-8,388,608 \mu\text{V}$ (-2^{23}) - 最大值: $8,388,607 \mu\text{V}$ ($2^{23} - 1$) - 分辨率: $1 \mu\text{V}$

4. 时间戳处理

4.1 传感器时间戳

Polar H10使用内部时间系统，需要转换为标准Epoch时间：

```
def convert_timestamp(sensor_timestamp, time_offset):
    """转换传感器时间戳为Epoch时间"""
    if time_offset is None:
        # 首次接收时计算偏移量
        time_offset = time.time_ns() - sensor_timestamp

    epoch_timestamp = sensor_timestamp + time_offset
    return epoch_timestamp, time_offset
```

4.2 时间戳特性

- **精度:** 纳秒级
- **参考点:** 数据帧最后一个样本的时间
- **同步:** 通过偏移量与系统时间同步

5. 实际数据示例

5.1 示例数据帧

原始十六进制数据:

```
00 15 81 E9 7D F4 10 22 11 00 E8 03 00 0C FE FF D0 07 00 24 FA
FF 20 03 00
```

字段解析:

- 字节0: 00 (ECG测量类型)
- 字节1-8: 15 81 E9 7D F4 10 22 11 (时间戳: 1234567890123456789)
- 字节9: 00 (ECG帧类型)
- 字节10-12: E8 03 00 (样本1: 1000 μV)
- 字节13-15: 0C FE FF (样本2: -500 μV)
- 字节16-18: D0 07 00 (样本3: 2000 μV)
- 字节19-21: 24 FA FF (样本4: -1500 μV)
- 字节22-24: 20 03 00 (样本5: 800 μV)

5.2 解码结果

```
{
  'measurement_type': 0,
  'timestamp': 1234567890123456789,
  'frame_type': 0,
  'sample_count': 5,
  'ecg_samples': [1000, -500, 2000, -1500, 800]
}
```

6. 控制命令协议

6.1 命令格式

控制命令通过PMD控制特征值发送:

```
# 操作码定义
GET_SETTINGS = 0x01    # 查询可用设置
START_STREAM  = 0x02    # 开始数据流
```

```
STOP_STREAM = 0x03      # 停止数据流
```

```
# 测量类型码
```

```
ECG_TYPE = 0x00         # ECG测量
```

6.2 开始ECG流命令

```
def create_start_ecg_command(sample_rate=130, resolution=14):  
    """创建开始ECG流的命令"""  
    cmd = bytearray([  
        0x02, # START命令  
        0x00, # ECG类型  
        0x00, # SAMPLE_RATE设置索引  
        0x01, # 参数数量  
        sample_rate & 0xFF, (sample_rate >> 8) & 0xFF, # 采样率  
        0x01, # RESOLUTION设置索引  
        0x01, # 参数数量  
        resolution & 0xFF, (resolution >> 8) & 0xFF # 分辨率  
    ])  
    return cmd
```

7. 错误处理

7.1 错误代码表

```
ERROR_CODES = {  
    0: 'SUCCESS',  
    1: 'INVALID OP CODE',  
    2: 'INVALID MEASUREMENT TYPE',  
    3: 'NOT SUPPORTED',  
    4: 'INVALID LENGTH',  
    5: 'INVALID PARAMETER',  
    6: 'ALREADY IN STATE',  
    7: 'INVALID RESOLUTION',  
    8: 'INVALID SAMPLE RATE',  
    9: 'INVALID RANGE',  
    10: 'INVALID MTU',  
    11: 'INVALID NUMBER OF CHANNELS',  
    12: 'INVALID STATE',  
    13: 'DEVICE IN CHARGER'  
}
```

7.2 常见错误处理

```
def handle_pmd_response(response):
    """处理PMD控制点响应"""
    if len(response) < 4:
        raise ValueError("响应太短")

    error_code = response[3]
    if error_code != 0:
        error_msg = ERROR_CODES.get(error_code, f"未知错误: {error_code}")
        raise RuntimeError(f"PMD错误: {error_msg}")

    return response
```

8. 性能考虑

8.1 数据传输速率

- 采样率: 130 Hz
- 样本大小: 3字节
- 理论带宽: 390字节/秒 (仅ECG数据)
- 实际帧开销: 10字节帧头 + 可变数据负载

8.2 缓冲区管理

```
# 推荐队列大小计算
def calculate_queue_size(sample_rate=130, buffer_seconds=5):
    """计算推荐的队列大小"""
    frames_per_second = sample_rate / 10 # 假设每帧10个样本
    return int(frames_per_second * buffer_seconds)
```

9. 实现示例

9.1 完整的ECG数据接收器

```
import asyncio
from bleak import BleakClient, BleakScanner

class PolarECGReceiver:
    def __init__(self):
        self.ecg_data = []
```

```

self.time_offset = None

async def connect_and_stream(self, device_address):
    """连接设备并开始ECG流"""
    async with BleakClient(device_address) as client:
        # 启动通知
        await client.start_notify(
            "FB005C82-02E7-F387-1CAD-8ACD2D8DF0C8",
            self.ecg_data_handler
        )

        # 发送开始命令
        start_cmd = bytearray([0x02, 0x00]) # START ECG
        await client.write_gatt_char(
            "FB005C81-02E7-F387-1CAD-8ACD2D8DF0C8",
            start_cmd
        )

        # 等待数据
        await asyncio.sleep(10)

        # 停止流
        stop_cmd = bytearray([0x03, 0x00]) # STOP ECG
        await client.write_gatt_char(
            "FB005C81-02E7-F387-1CAD-8ACD2D8DF0C8",
            stop_cmd
        )

def ecg_data_handler(self, sender, data):
    """ECG数据处理"""
    try:
        ecg_samples = self.decode_ecg_frame(data)
        self.ecg_data.extend(ecg_samples)
        print(f"接收到 {len(ecg_samples)} 个ECG样本")
    except Exception as e:
        print(f"解码错误: {e}")

def decode_ecg_frame(self, data):
    """解码ECG数据帧"""
    if len(data) < 10:
        raise ValueError("数据帧太短")

    if data[0] != 0x00:
        raise ValueError("不是ECG数据帧")

    if data[9] != 0x00:
        raise ValueError("无效的ECG帧类型")

    # 解析时间戳
    timestamp = int.from_bytes(data[1:9], 'little',
signed=False)
    if self.time_offset is None:

```

```
        self.time_offset = time.time_ns() - timestamp

# 解析ECG样本
payload_length = len(data) - 10
if payload_length % 3 != 0:
    raise ValueError("ECG数据长度错误")

samples = []
for i in range(0, payload_length, 3):
    offset = 10 + i
    sample = int.from_bytes(
        data[offset:offset+3],
        'little',
        signed=True
    )
    samples.append(sample)

return samples
```

10. 总结

Polar H10的ECG数据格式是一个精心设计的二进制协议，具有以下特点：

1. **高效编码**: 使用3字节编码实现微伏级精度
2. **时间同步**: 提供纳秒级时间戳精度
3. **错误检测**: 内置帧类型和长度验证
4. **实时传输**: 支持130Hz实时数据流
5. **标准化**: 遵循BLE规范和Polar PMD协议

该格式适用于各种ECG应用场景，从基础心率监测到高精度心电图分析。通过理解其底层数据结构，开发者可以构建高效、可靠的ECG数据处理应用。

文档版本: 1.0

最后更新: 2025年6月8日

基于: bleakheart v0.1.3 源码分析